

A Zero-Knowledge Proof-Based Protocol for Privacy-Preserving Transactions on Public Blockchains

Hao Pham-Duc
hao.pham1652003@hcmut.edu.vn
HCMC University of Technology
(HCMUT), Vietnam
Vietnam National University-HCMC,
Vietnam

Khang Vo-Vi
vovikhangcdv@gmail.com
Ho Chi Minh City, Vietnam

Khuong Nguyen-An*
nakhuong@hcmut.edu.vn
HCMC University of Technology
(HCMUT), Vietnam
Vietnam National University-HCMC,
Vietnam

Abstract

Public blockchains offer transparency and decentralization but also expose sensitive transaction data that compromise user privacy. This issue can be considered as a data breach if traditional financial systems attempt to migrate their infrastructures onto a public blockchain, creating a legal barrier to blockchain adoption in practical fields. Existing solutions often require restructuring the blockchain architecture, relying on protocols with high communication and computation costs, or launching a separate blockchain network, which can erode efficiency and undermine the decentralization of blockchain. In this paper, we propose a protocol that allows users to carry out privacy-preserving transactions on public blockchains. Our protocol leverages homomorphic encryption to encrypt user balances and employs commitment scheme to store the entire ledger on-chain compactly and efficiently. By integrating succinct zero-knowledge proofs, we enable the verification of state transitions (such as deposits, withdrawals, and fund transfers) without disclosing confidential inputs such as private keys, senders, receivers, or transaction amounts. We also present a concrete construction by selecting specific cryptographic primitives and analyze its security to demonstrate privacy and correctness. Finally, we conduct experiments on the proposed construction, measuring its performance to show that it can achieve both efficiency and scalability on an EVM-based blockchain.

CCS Concepts

• **Security and privacy** → *Pseudonymity, anonymity and untraceability*; **Privacy-preserving protocols**.

Keywords

Blockchain, Privacy, Anonymity, Zero-Knowledge Proof

ACM Reference Format:

Hao Pham-Duc, Khang Vo-Vi, and Khuong Nguyen-An. 2025. A Zero-Knowledge Proof-Based Protocol for Privacy-Preserving Transactions on Public Blockchains. In *Proceedings of the 7th ACM International*

*Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
BSCI '25, Hanoi, Vietnam

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1412-2/2025/08
<https://doi.org/10.1145/3709016.3737794>

Symposium on Blockchain and Secure Critical Infrastructure (BSCI '25). ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3709016.3737794>

1 INTRODUCTION

Since the day Bitcoin was created by an anonymous entity named Satoshi Nakamoto in 2009 [14], blockchain technology has become a potential game-changer in finance, supply chains, and identity management. It eliminates the need for a trusted third party by having a network of independent nodes verify data. However, public blockchains must store transaction data transparently, including amounts, sender identities, and recipient addresses, raising privacy concerns for traditional organizations. Many financial institutions hesitate to adopt these systems due to fears of violating regulations, such as the General Data Protection Regulation (GDPR), which allows individuals to request data deletion (Article 17). GDPR's principle of "purpose limitation" also restricts data access to authorized parties, conflicting with blockchain's transparency. Research shows that over two-thirds of blockchain projects fail to address GDPR compliance [3], creating significant barriers for industries transitioning from legacy systems to public blockchains.

To address these privacy and compliance issues, we propose a protocol that maintains transaction confidentiality on public blockchains. It utilizes a minimal encrypted ledger for user balances, with its state root in the smart contract storage. State transitions use zero-knowledge proofs (ZKP) to keep sensitive data private. Our protocol is generic, allowing for different cryptographic primitive settings. This work has three main contributions:

- We propose a privacy-preserving protocol using homomorphic encryption, commitments, and zkSNARK to help users maintain privacy when interacting with public blockchains.
- We instantiate our protocol with a concrete, selective choice of cryptographic primitives, including Exponential ElGamal [1], KZG [11], and Groth16 [10].
- We benchmark our prototype in the EVM environment, demonstrating that while off-chain proof generation is the main bottleneck, on-chain computational costs are significantly lower than Aztec [19] and Tornado Cash [17].

2 RELATED WORK

One of the early methods to obscure on-chain transaction links is CoinJoin, introduced by Maxwell [12]. This technique merges multiple inputs and outputs into a single transaction, making it hard to determine which input corresponds to which output. Another method involves ring signatures, developed by Rivest et

al. [18]. However, these techniques often require an additional communication layer to gather mixing partners, making them vulnerable to Sybil attacks [2] if an adversary creates multiple addresses. Additionally, as noted by Maxwell, these methods can be susceptible to Denial of Service (DoS) attacks, such as refusing to sign transactions or spending coins before completion [12].

A straightforward way to address issues with permissionlessness is to use permissioned blockchains, where only authorized parties can access transaction data [7, 8]. Hyperledger Fabric [4] offers a framework for enterprises, requiring participants to register through a trusted Membership Service Provider and supporting private transactions that only share hashed or encrypted data with selected nodes. ZCash [5], a Bitcoin fork, incorporates zero-knowledge proofs (ZKPs) to provide two types of addresses: Transparent addresses (t-addresses), which lack privacy, and Shielded addresses (z-addresses), which encrypt transaction data while allowing verification by network. Transactions in ZCash can be categorized into four types based on privacy levels: Transparent transactions, Shielding transactions, Deshielding transactions, and Shielded transactions. However, a drawback is that transaction data remains visible to a group of authorized miners, and transferring assets from a public blockchain may require a bridge mechanism, introducing security risks.

Another approach involves using smart contracts on EVM-based public blockchains without client modifications. Tornado Cash [17] utilizes a Merkle Tree [13] for fixed-denomination ETH deposits, where each leaf is a hashed secret. Withdrawers prove knowledge of an unspent leaf's preimage using ZKPs, specifically Groth16 [10], which avoids on-chain depositor-withdrawer associations. However, Tornado Cash has limitations, such as fixed deposit and withdrawal amounts, and functions as a one-way protocol, making it seem more like a money laundering tool than a privacy solution for legitimate users. AZTEC [19] addresses Tornado Cash's fixed deposit issue using a UTXO-like model [14] with "notes" for confidential transfers through join-split transactions. While it allows users to prove the equality of input and output notes without revealing decryption keys, its on-chain costs are high, with join-split transactions costing millions gas. AZTEC supports auditability but lacks the ability for users to control the viewing key, leaving the third party with permanent access to the note's value. Both projects appear inactive, highlighting practical challenges in privacy solutions.

3 PROPOSED PROTOCOL

This section presents the design goals and protocol design, abstracting the cryptographic primitives first and instantiating them later to illustrate flexible trade-offs among computation, cost, and security.

3.1 Key Required Features

Our protocol is designed to allow privacy-preserving transactions on public blockchains. To achieve this, we define the following core design goals:

- **Functionality:** The protocol allows users to deposit, withdraw, and transfer funds to others.

- **Privacy:** The protocol enables hiding transaction details, including sender, receiver, and transferred amount.
- **Recoverability:** The protocol lets users share their balances with others and revoke this permission at anytime.
- **Integration:** The smart contracts of our protocol should be compatible with EVM-based blockchains.
- **On-chain Efficiency:** The protocol should minimize computation and storage costs on the chain.

3.2 Overview of Proposed Protocol

Our protocol uses a minimal ledger to record user balances, updated with each transaction. To safeguard these balances from unauthorized access, we employ an **additively homomorphic encryption scheme**, allowing updates without decryption. Instead of storing all records on the Blockchain, which is inefficient, we utilize a **commitment scheme** that compresses records into a single commitment stored in a smart contract. This approach still permits verification of individual records through opening proofs.

The smart contract holds all encrypted records reflecting the current state of a state machine $\mathcal{S}$, denoted as ϕ . The state ϕ updates with user actions such as deposits, withdrawals, fund transfers, or key rotations. The state at time t is represented as ϕ_t . At any given time, the state machine $\mathcal{S}$ can perform one of four actions to transition to a new state.

- **deposit:** Invoked when a user registers a public key and deposits funds, it adds a new encrypted balance entry to the ledger.
- **withdraw:** Invoked when a user withdraws funds, it homomorphically subtracts the amount from the encrypted balance without decryption.
- **send:** Invoked when a user transfers funds, updating the sender's and receiver's encrypted balances.
- **rotate:** Invoked when a user updates their viewing key for their encrypted balance, revoking third-party access and ensuring future privacy.

Each action triggers an update from state ϕ_t to ϕ_{t+1} . To ensure efficient updates, our commitment scheme should utilize additive homomorphism, allowing updates with just a few additions to ϕ_t .

State transitions in $\mathcal{S}$ are represented by a program $\mathcal{P}$ that takes the current state ϕ_t and necessary inputs to output the next state ϕ_{t+1} . Verifying the correctness of our protocol involves confirming that $\mathcal{P}$ produces the correct output at time t . By employing zkSNARKs, we can validate this execution efficiently while preserving user privacy, as they do not require all inputs. Furthermore, zkSNARKs allow us to delegate program execution to off-chain, with the smart contract only needing to verify the proof and update the state, thus significantly reducing on-chain operational costs.

Figure 1 shows the interaction flow among three main entities:

- **User (U):** Performs actions (deposit, withdraw, send, rotate).
- **Trusted Backend (TB):** Manage off-chain storage, computes state transitions, and generates proofs according to user requests.
- **Blockchain (BC):** Stores the smart contract, verifies proofs, and maintains the current protocol state.

An optional entity called a **Relayer** can submit transactions on behalf of users, which is beneficial when users lack sufficient funds

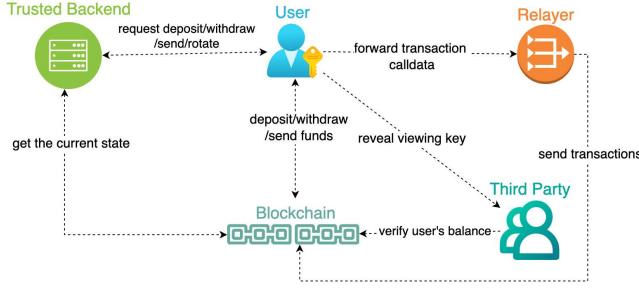


Figure 1: Overview of the proposed protocol.

for transaction fees (gas). Formally, our protocol consists of ten algorithms defined as follows, clearly indicating the executor in each subscript:

- $\text{SetUp}_{\text{TB}}(\lambda) \rightarrow (\text{pp}, \text{rec})$: Initializes public parameters pp for cryptographic primitives and an initial empty record rec of protocol requests, updated after each user interaction.
- $\text{KeyGen}_{\text{U}}(\text{pp}) \rightarrow (\text{sk}, \text{pk})$: Generates a user's private/public key pair (sk, pk) for balance encryption.
- $\text{ProveDeposit}_{\text{TB}}(\text{pp}, \text{rec}, \text{pk}, r, x, \phi_t) \rightarrow (\text{rec}', \phi_{t+1}, \pi_d)$: Upon receiving a user's deposit request with public key pk , randomness r , and deposit amount x , computes the next state ϕ_{t+1} and produces the proof π_d .
- $\text{ProveWithdraw}_{\text{TB}}(\text{pp}, \text{rec}, \text{sk}, x, \text{rcp}, \phi_t) \rightarrow (\text{rec}', \phi_{t+1}, \pi_w)$: Upon receiving a withdrawal request containing the user's secret key sk , withdrawal amount x , and recipient address rcp , computes the next state ϕ_{t+1} and generates proof π_w .
- $\text{ProveSend}_{\text{TB}}(\text{pp}, \text{rec}, \text{sk}_s, \text{pk}_r, x, \phi_t) \rightarrow (\text{rec}', \phi_{t+1}, \pi_s)$: Given a send request containing sender's secret key sk_s , receiver's public key pk_r , and amount x , computes the next state ϕ_{t+1} and generates the proof π_s .
- $\text{ProveRotate}_{\text{TB}}(\text{pp}, \text{rec}, \text{sk}, r', \phi_t) \rightarrow (\text{rec}', \phi_{t+1}, \pi_r)$: When a user requests to rotate their viewing key with a new randomness r' , computes the updated state ϕ_{t+1} and creates a corresponding proof π_r .
- $\text{VerifyDeposit}_{\text{BC}}(\text{pp}, \phi_t, \phi_{t+1}, x, \text{pk}, \pi_d) \rightarrow \{0, 1\}$: Verifies the deposit proof π_d , ensuring correct commitment update based on the provided deposit amount x and user's public key pk .
- $\text{VerifyWithdraw}_{\text{BC}}(\text{pp}, \phi_t, \phi_{t+1}, x, \text{rcp}, \pi_w) \rightarrow \{0, 1\}$: Verifies the withdraw proof π_w , confirming the correct commitment update corresponding to the withdrawal amount x and recipient address rcp .
- $\text{VerifySend}_{\text{BC}}(\text{pp}, \phi_t, \phi_{t+1}, \pi_s) \rightarrow \{0, 1\}$: Checks the correctness of send proof π_s , ensuring valid transfers between users.
- $\text{VerifyRotate}_{\text{BC}}(\text{pp}, \phi_t, \phi_{t+1}, \pi_r) \rightarrow \{0, 1\}$: Validates the rotate proof π_r , verifying that the user's encrypted balance randomness update is correct.

Figure 2 illustrates the process of a **User** depositing funds into our protocol. The **User** starts by generating a public/private key pair using the KeyGen algorithm and sends a request to **TB** for a deposit action. The backend queries the current state of the contract on the **Blockchain** and executes the program $\mathcal{P}$ based on the provided information to compute the next state and generate proof using

the ProveDeposit algorithm. This proof and public values are then sent back to the **User**. Finally, the **User** calls the $\text{deposit}()$ function on the smart contract with the proof, public values, the new state ϕ_{t+1} , and the deposit amount x .

On the **Blockchain**, upon receiving a transaction from the **User**, the VerifyDeposit algorithm verifies the proof through a verifier contract. If valid, it checks the user's fund amount. Upon successful verification, the smart contract emits the **Deposited()** event to **TB**, which updates its records.

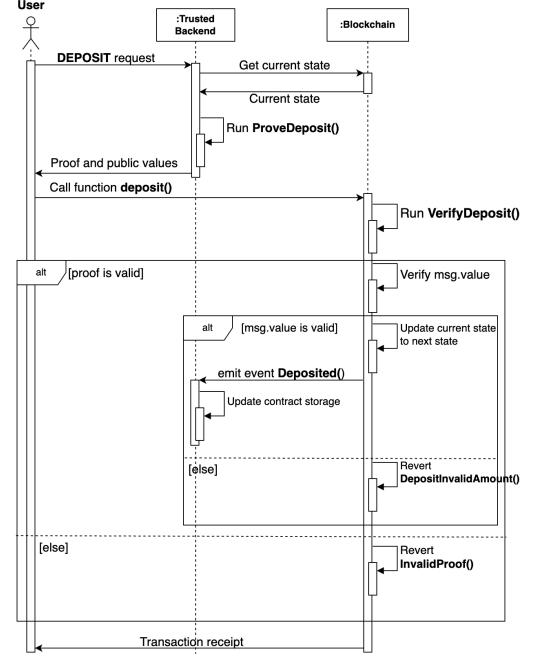


Figure 2: Sequence diagram of the deposit process.

Figure 3 outlines how a **User** withdraws funds from the protocol. They initiate a **WITHDRAW** request to **TB**, specifying a recipient address and necessary parameters. After obtaining proof and public values from the backend, these are forwarded to a **Relayer**, which then calls the $\text{withdraw}()$ function on the smart contract. The contract verifies the proof using the VerifyWithdraw algorithm and transfers the funds to the recipient address, emitting the **Withdrew()** event to inform **TB**. Access to the proof and public values makes the process vulnerable to front-running attacks; however, the recipient address is included as a redundancy constraint, ensuring any changes make the proof invalid and protect the user from such attacks.

Figure 4 illustrates the process of transferring funds between users within the protocol. The sender must know the recipient's identity and public key to send funds correctly. After obtaining the necessary proof and values, the sender can execute the send action directly or through a **Relayer** for added privacy. Once the transaction is successful, an event **Sent()** is emitted on the blockchain, allowing **TB** to update its database. It's important to note that the total funds locked in the smart contract remain unchanged throughout this process.

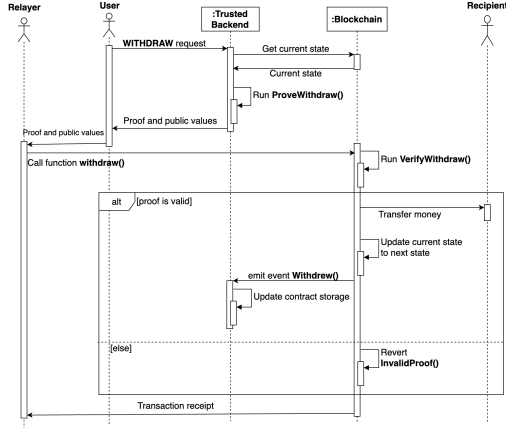


Figure 3: Sequence diagram of the withdrawal process.

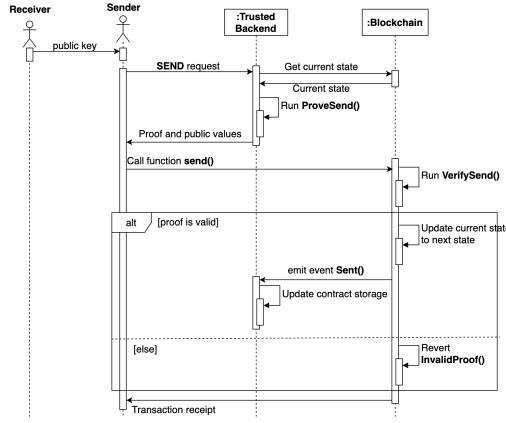


Figure 4: Sequence diagram of the fund transfer process within the protocol.

Users may want to reveal their balance without withdrawing all funds. Revealing plaintext encrypted with probabilistic encryption doesn't require compromising the private key; only the randomness needs to be disclosed, which is unique for each message. It's essential to allow users to revoke this permission to keep their balance private. Many homomorphic cryptosystems enable modifying the randomness using their properties. For instance, in the EE cryptosystem, where E is the encryption function, we have the following useful property

$$\begin{aligned} E(m, r_1) \times E(0, r_2) &= (g^{r_1}, g^m \cdot pk^{r_1}) \times (g^{r_2}, g^0 \cdot pk^{r_2}) \\ &= (g^{r_1+r_2}, g^m \cdot pk^{r_1+r_2}) = E(m, r_1 + r_2). \end{aligned}$$

Users can rotate their randomness from r_1 to $r_1 + r_2$ without re-encrypting the balance. This allows our protocol to help users grant and revoke balance verification permissions, which we will refer to as the user's viewing key.

Figure 5 shows the key rotation process. The **User** first reveals their balance and randomness to a third party, who verifies this via an on-chain commitment. To revoke this access, the **User** requests a ROTATE action from **TB**, providing new randomness. They then

call the `rotate()` function on the smart contract, which verifies the proof and updates the commitment before emitting a **Rotated()** event. Consequently, the third party can no longer verify the **User**'s balance.

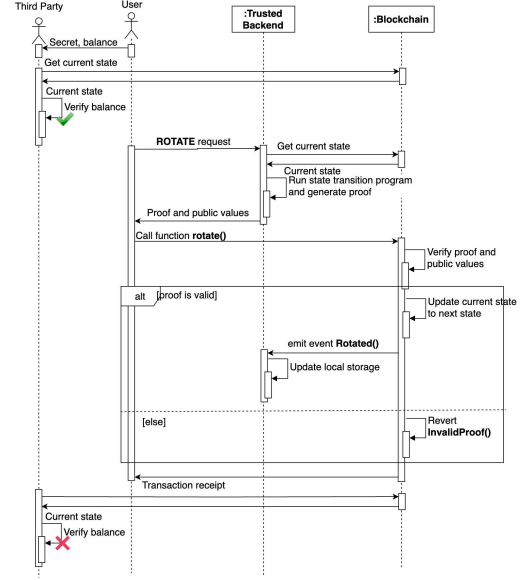


Figure 5: Sequence diagram of the viewing key rotation process.

3.3 A Concrete Construction

In this section, we provide a concrete instantiation of the protocol introduced earlier, discussing trade-offs among three schemes for each cryptographic component and selecting the best one based on specific criteria.

We evaluate three additively homomorphic encryption schemes against four criteria in Table 1. CKKS [6] supports complex numbers but is deemed "not ZK-friendly" due to its constraints. Paillier [15], based on the decisional composite residuosity assumption, requires a modulus n of at least the product of two 1024-bit primes, making it inefficient for zkSNARK circuits. In contrast, Exponential ElGamal (EE) [1], which operates on finite fields, integrates well with zkSNARK circuits when parameters are chosen correctly. Consequently, we selected EE as our encryption scheme.

Table 1: Comparison of three homomorphic encryption schemes: CKKS, Paillier, and EE.

Scheme	Homomorphic	Plaintext domain	Ciphertext domain	ZK-Friendly
CKKS [6]	Fully	$\mathbb{C}$	$(\mathbb{Z}_p[X]/(X^n + 1))^2$	$\times$
Paillier [15]	Additive	$\mathbb{Z}_n$	$\mathbb{Z}_{n^2}$	$\times$
EE [1]	Additive	$\mathbb{F}_p$	$\mathbb{F}_p^2$	$\checkmark$

For the commitment scheme, we evaluate three candidates and five criteria in Table 2. While all have constant-size commitments, Merkle Tree [13] incurs logarithmic proof size and verification time, leading to higher on-chain operational costs as user numbers grow.

The homomorphic property reduces the commitment update cost, while selective opening enables smart contracts to verify specific records without recomputing the entire commitment. We select KZG [11] as our commitment scheme based on these criteria.

Table 2: Comparison of three commitment schemes: Pedersen, Merkle Tree, and KZG.

Scheme	Commitment size	Proof size	Verification time	Selective opening	Homomorphic
Pedersen [16]	$O(1)$	$O(1)$	$O(1)$	✗	✓
Merkle Tree [13]	$O(1)$	$O(\log n)$	$O(\log n)$	✓	✗
KZG [11]	$O(1)$	$O(1)$	$O(1)$	✓	✓

For the zero-knowledge proving system, we focus on proof size and verification cost, as shown in Table 3 for two candidates: Plonk [9] and Groth16 [10]. Groth16 has smaller proof size and lower on-chain verification costs but requires a trusted setup for each circuit. Since the circuit, $\mathcal{P}$, is fixed, the trusted setup can be conducted once and reused for all users, making Groth16 the preferred option.

Table 3: Comparison of two zkSNARK proving systems: Plonk, Plonk (fast prover version), and Groth16.

Scheme	Proof size (bytes)	Verification cost (gas)	Trusted setup
Plonk [9]	640	≈ 223000	universal
Plonk (fast prover)	768	≈ 300000	universal
Groth16 [10]	256	≈ 200000	per-circuit

We represent each user's balance as ciphertext using the EE scheme. Each user generates a secret key sk and a public key $pk = g^{sk}$. The balance x is encrypted as $(g^r, g^x \cdot pk^r)$ for a random r . These ciphertexts are stored in a d -length vector $\mathbf{v}$, while a KZG commitment succinctly represents it on-chain. We encode $\mathbf{v}$ as a polynomial $P_v(x)$ such that $P_v(\omega^i) = v_i$ and commit to it by evaluating $P_v(x)$ at a secret point τ . The group element $P_v(\tau)G$ acts as the state of program $\mathcal{P}$, denoted ϕ . Any change in user balances updates $\mathbf{v}$ and $P_v(x)$ in a way that can be proven via a zero-knowledge argument.

The remaining of this section will describe the overview processes of each action in the protocol under this concrete construction. These processes will be realized by the algorithms defined in Section 4.

When a user deposits x coins, they select a new index i and encrypt the amount with their public key pk . The backend updates the polynomial P_v so that $P_v(\omega^i) = g^x \cdot pk^r$ and computes the corresponding commitment. Finally, it prepares a Groth16 proof to verify the update's consistency.

Withdrawals follow a reverse pattern. The backend identifies the ciphertext for the user's balance by locating the index i for their public key. It checks if the balance can cover the withdrawal, then modifies the ciphertext to reduce x coins by adjusting the committed polynomial at position i . This change is proven in zero-knowledge, ensuring only the secret key holder can alter the ciphertext and that the on-chain state aligns with the withdrawal. Once verified, the contract releases the x coins to the recipient address and updates the protocol's commitment.

Transfers between users utilize homomorphic properties to adjust two ciphertexts: decreasing the sender's balance and increasing the recipient's. The user generating the proof confirms knowledge of the sender's secret key, ensuring the transferred amount aligns with the sender's balance and the new recipient ciphertext.

At the time t , if a user, whose encrypted balance is v_i and secret is r , wants to reveal their balance to a third party $\mathcal{T}$, they can issue the trusted backend an opening proof π , which proves that $P_v(\omega^i) = v_i$. They send (π, v_i, pk, r, x) to $\mathcal{T}$ to claim that their balance is x . At a time $t' > t$, $\mathcal{T}$ can verify this claim by doing the following steps:

- (1) Check if g^r match the on-chain value.
- (2) Check if $g^x \times pk^r \stackrel{?}{=} v_i$.
- (3) Perform KZG proof verification by checking this equation

$$e(\pi, (\tau - \omega^i)G_2) + e(\phi_{t'} - \phi_t, G_2) \stackrel{?}{=} e(\phi_{t'} - v_i G_1, G_2),$$

where $e(\cdot, \cdot)$ is the bilinear pairing function and G_1, G_2 are the generators of the elliptic curve groups generated in the KZG setup.

We will explain the third step as follows. Let $P_v(x)$ and $P'_v(x)$ be the polynomials corresponding to ϕ_t and $\phi_{t'}$, respectively. If $P_v(\omega^i) = P'_v(\omega^i) = v_i$, then the proof π' for $P'_v(\omega^i) = v_i$ can be computed as

$$\pi' = \frac{P'_v(\tau) - v_i}{\tau - \omega^i} G_1 = \pi + \frac{P'_v(\tau) - P_v(\tau)}{\tau - \omega^i} G_1.$$

To verify π' , one can compute the bilinear $e(\pi', (\tau - \omega^i)G_2)$ as

$$\begin{aligned} e(\pi', (\tau - \omega^i)G_2) &= e(\pi, (\tau - \omega^i)G_2) + e((P'_v(\tau) - P_v(\tau))G_1, G_2) \\ &= e(\pi, (\tau - \omega^i)G_2) + e(\phi_{t'} - \phi_t, G_2). \end{aligned}$$

The protocol supports a "secret rotation" step, allowing users to change the randomness encrypting their balance after revealing it to a trusted party. This is done by multiplying the ciphertext by an off-chain blinding term, revoking previous visibility. It also involves updating the polynomial commitment and generating proof for on-chain verification.

In summary, these actions illustrate how a private transaction framework can be achieved: EE ensures balance confidentiality, KZG commitments represent an entire ledger in one group element, and Groth16 guarantees verified on-chain updates without revealing sensitive information. This combination proves the protocol's feasibility on a public blockchain.

4 EXPERIMENTS AND EVALUATION

This section outlines the experimental setup and implementation of our proposed protocol, aiming to evaluate its feasibility and performance. The source code will be available upon reasonable request.

4.1 Development Environment

We implement **TB** with Rust (rustc 1.81.0 (eeb90cda1 2024-09-04)) and Solidity (v0.8.23+commit.f704f362). Our experiments were conducted on a MacBook M1 Pro with 16 GB RAM. The operating system is macOS Sequoia 15.3. The Ethereum networks used in this experiment are local networks (Shanghai hard fork) and Optimism. We use Anvil (v0.3.0) to fork the state of Optimism from block 131843560. The deployment scripts and test cases are simulated using Foundry (v0.3.0).

4.2 Implementaion Details

The implementation consists of two main modules: off-chain and on-chain. In the off-chain module, we implement the state transition logic based on the construction described in 3.2. This logic is compiled into an ELF file for Succint RISC-V zkVM using the SP1 framework (v4.1.0). The off-chain module manages state transitions and generates proofs. It is stateful, maintaining all records for reconstructing $\mathcal{S}$ to its latest state. The following paragraphs outline how each action is performed.

The program is configured for a maximum of deg users, matching the highest degree of committable polynomials. It initializes a counter cnt to track user numbers and two deg -length arrays, $\mathbf{t}$ for encrypted randomness and $\mathbf{v}$ for encrypted balances. A hash map index_of maps each user's public key to its index, optimizing lookups from $O(n)$ to $O(1)$. Initially, cnt , $\mathbf{t}$, and $\mathbf{v}$ are set to zero, making ϕ_0 the commitment of $P(x) \equiv 0$.

We omit the detailed proof generation process, as the SP1 framework manages it automatically. Instead, we focus on state computation through four actions: deposit, withdraw, send, and rotate. Algorithms 1, 2, 3, and 4 outline the logic for these actions in the off-chain module. After each execution, a computational trace is generated and sent to the SP1 prover engine for proof generation. These processes are ProveDeposit, ProveWithdraw, ProveSend, and ProveRotate, respectively.

Algorithm 1 (DEPOSIT) encrypts the deposit amount and updates the commitment using a Lagrange-basis-scaled term from the ciphertext, leveraging the additively homomorphic property of the KZG commitment scheme. Conversely, Algorithm 2 (WITHDRAW) verifies the sender's balance, checks the validity of the requested amount, and then homomorphically subtracts that amount from the commitment. While balance verification can use a range proof to avoid decryption, we simplify by decrypting the ciphertext to compare the balance with the requested amount.

Algorithm 1 DEPOSIT state transition

```

1: function DEPOSIT(pkey, r, amount,  $\phi_t$ )
2:   if  $\text{cnt} \geq \text{deg}$  then
3:     return FAIL("Total user exceed limit")
4:   end if
5:    $(t, v) \leftarrow \text{Enc}(pkey, \text{amount}, r)$ 
6:    $t_{\text{cnt}} \leftarrow t$ 
7:    $v_{\text{cnt}} \leftarrow v$ 
8:   if  $\text{index\_of}[pkey]$  exists then
9:     return FAIL("Public key already exists")
10:  end if
11:   $\text{index\_of}[pkey] \leftarrow \text{cnt}$ .
12:   $\phi_{t+1} \leftarrow \phi_t + \ell_{\text{cnt}} \times v$  ▷  $\ell_{\text{cnt}}$  is the cnt-th Lagrange basis
13:   $\text{cnt} \leftarrow \text{cnt} + 1$ .
14:  return  $\phi_{t+1}$ 
15: end function

```

Algorithm 3 (SEND) manages the send action between users by checking the sender's balance and adjusting it accordingly. Algorithm 4 (ROTATE) handles key rotation by multiplying the user's ciphertext with a zero-value ciphertext, ensuring the balance remains unchanged. The four actions are merged into a single program that uses branch logic to execute the user-requested operation, shrinking contract storage from four verification keys to just one for the main program.

The on-chain module includes a smart contract that updates the state of $\mathcal{S}$ upon validating proofs and public values from

Algorithm 2 WITHDRAW state transition

```

1: function WITHDRAW(skey, balance, amount, recipient,  $\phi_t$ )
2:   pkey  $\leftarrow g^{\text{skey}}$ 
3:   if  $\text{index\_of}[pkey]$  does not exist then
4:     return FAIL("Public key not found")
5:   end if
6:    $i \leftarrow \text{index\_of}[pkey]$ 
7:    $m \leftarrow \text{Dec}(skey, (t_i, v_i), \text{balance})$ .
8:   if  $\text{amount} > m$  then
9:     return FAIL("Insufficient amount")
10:  end if
11:   $\phi_{t+1} \leftarrow \phi_t + \ell_i \times v_i \times (g^{-\text{amount}} - 1)$  ▷  $\ell_i$  is the  $i$ -th Lagrange basis
12:   $v_i \leftarrow v_i \times g^{-\text{amount}}$ 
13:  tmp  $\leftarrow$  recipient ▷ Redundancy to commit recipient
14:  return  $\phi_{t+1}$ 
15: end function

```

Algorithm 3 SEND State Transition

```

1: function SEND(skey_sender, pkey_receiver, balance, amount,  $\phi_t$ )
2:   pkey_sender  $\leftarrow g^{\text{skey\_sender}}$ 
3:   if  $\text{index\_of}[pkey\_sender]$  does not exist then
4:     return FAIL("Sender public key not found")
5:   end if
6:   if  $\text{index\_of}[pkey\_receiver]$  does not exist then
7:     return FAIL("Receiver public key not found")
8:   end if
9:    $i_s \leftarrow \text{index\_of}[pkey\_sender]$ 
10:   $i_r \leftarrow \text{index\_of}[pkey\_receiver]$ 
11:   $m \leftarrow \text{Dec}(skey\_sender, (t_{i_s}, v_{i_s}), \text{balance})$ 
12:  if  $\text{amount} > m$  then
13:    return FAIL("Insufficient balance")
14:  end if
15:   $P_s \leftarrow \ell_{i_s} \times v_{i_s} \times (g^{-\text{amount}} - 1)$ 
16:   $P_r \leftarrow \ell_{i_r} \times v_{i_r} \times (g^{\text{amount}} - 1)$ 
17:   $\phi_{t+1} \leftarrow \phi_t + P_s + P_r$ 
18:   $v_{i_s} \leftarrow v_{i_s} \times g^{-\text{amount}}$ 
19:   $v_{i_r} \leftarrow v_{i_r} \times g^{\text{amount}}$ 
20:  return  $\phi_{t+1}$ 
21: end function

```

Algorithm 4 ROTATE State Transition

```

1: function ROTATE(skey,  $r'$ ,  $\phi_t$ )
2:   pkey  $\leftarrow g^{\text{skey}}$ 
3:   if  $\text{index\_of}[pkey]$  does not exist then
4:     return FAIL("Public key not found")
5:   end if
6:    $i \leftarrow \text{index\_of}[pkey]$ 
7:   $\phi_{t+1} \leftarrow \phi_t + \ell_i \times v_i \times (pkey^{r'} - 1)$ 
8:   $t_i \leftarrow t_i \times g^{r'}$ 
9:   $v_i \leftarrow v_i \times pkey^{r'}$ 
10:  return  $(\phi_{t+1}, i)$ 
11: end function

```

users. It stores the verification key, the Groth16 verifier contract address, the current state ϕ , and a mapping of public keys to encrypted randomness. Algorithms 5, 6, 7, and 8 outline the functions deposit, withdraw, send, and rotate for use by Users or Relayer.

4.3 Experiment Results

Table 4: Experiment results when $\text{deg} = 16$.

Action	Execution Time (ms)	Proving Time (s)	Proof Size (bytes)	Verifying Time (ms)	Gas (units)
Deposit	92.48	515.53	260	653.27	276395
Withdraw	170.85	654.87	260	669.24	244582
Send	199.21	901.56	260	642.66	209760
Rotate	90.83	640.11	260	586.98	209760

Algorithm 5 DEPOSIT on smart contract

```

1: function DEPOSIT(publicValues, proof)
2:   Call Groth16 verifier contract to verify proof. Revert if verification fails.
3:   Decode  $\phi_t, \phi_{t+1}, \text{amount}, \text{pkey}, t$  from publicValues.
4:   if  $\phi_t \neq \phi$  then
5:     return FAIL("Current state does not match")
6:   end if
7:   if amount  $\neq \text{msg.value}$  then
8:     return FAIL("Amount does not match")
9:   end if
10:   $\phi \leftarrow \phi_{t+1}$ 
11:  publicKeyToRandomness[pkey]  $\leftarrow t$ 
12:  Emit event Deposited().
13: end function

```

Algorithm 6 WITHDRAW on smart contract

```

1: function WITHDRAW(publicValues, proof)
2:   Call Groth16 verifier contract to verify proof. Revert if verification fails.
3:   Decode  $\phi_t, \phi_{t+1}, \text{amount}, \text{recipient}$  from publicValues.
4:   if  $\phi_t \neq \phi$  then
5:     return FAIL("current state does not match")
6:   end if
7:   if amount >  $t.\text{this.balance}$  then
8:     return FAIL("Insufficient balance")
9:   end if
10:  Transfer amount wei to recipient.
11:   $\phi \leftarrow \phi_{t+1}$ 
12:  Emit event Withdraw().
13: end function

```

Algorithm 7 SEND on smart contract

```

1: function SEND(publicValues, proof)
2:   Call Groth16 verifier contract to verify proof. Revert if verification fails.
3:   Decode  $\phi_t, \phi_{t+1}, \text{pkey}, t'$  from publicValues.
4:   if  $\phi_t \neq \phi$  then
5:     return FAIL("Current state does not match")
6:   end if
7:   $\phi \leftarrow \phi_{t+1}$ 
8:  Emit event Sent().
9: end function

```

Algorithm 8 ROTATE on smart contract

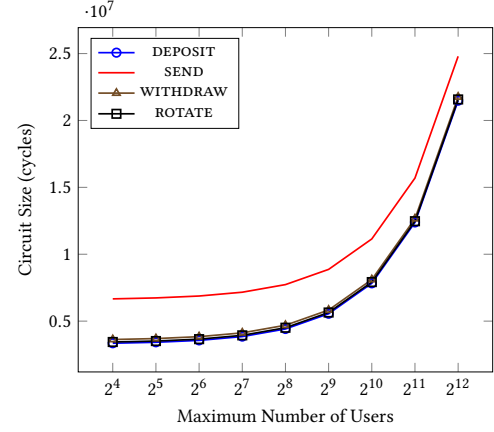
```

1: function ROTATE(publicValues, proof)
2:   Call Groth16 verifier contract to verify proof. Revert if verification fails.
3:   Decode  $\phi_t, \phi_{t+1}, \text{pkey}, t'$  from publicValues.
4:   if  $\phi_t \neq \phi$  then
5:     return FAIL("Current state does not match")
6:   end if
7:   if publicKeyToRandomness[pkey] does not exist then
8:     return FAIL("Unregistered public key")
9:   end if
10:  publicKeyToRandomness[pkey]  $\leftarrow t'$ 
11:   $\phi \leftarrow \phi_{t+1}$ 
12:  Emit event Rotated().
13: end function

```

Table 4 shows performance results for up to 16 users, with average times calculated over 100 runs. **Execution Time** (in milliseconds) increases with update complexity; the send action, which modifies two ciphertexts, has a notably higher time. **Proving Time** (in seconds) grows linearly with circuit size, with the send action taking the longest at 901.56 s due to its larger circuit. **Proof Size** remains at 260 bytes for all actions, demonstrating the efficiency of the Groth16 protocol. **Verifying Time** (in ms) and **Gas** (in units) for proof verification and smart contract updates are low and not affected by action complexity.

To assess scalability, we measure circuit size in zkVM cycles as user numbers increase. More cycles indicate greater complexity and longer proving times. Results from Figure 6 show that circuit size grows linearly with users, with the SEND circuit's complexity being

**Figure 6: Circuit size.**

the highest due to multiple addition operations on commitments. While verification time and on-chain costs remain stable, proving time increases linearly with circuit size, causing increased resource consumption for TB. To address this, we can deploy multiple protocol instances in parallel, using an intermediary protocol for cross-instance communication.

The results indicate that proving time is the main bottleneck of our protocol. This overhead can be reduced with specialized hardware accelerators or a decentralized proving network to parallelizes proof generation. On-chain gas costs remain high, but the proof size and verification process are efficient. Our current progress lays the groundwork for circuit optimization and hardware accelerators to lower off-chain operation costs.

4.4 Comparison

Table 5: Comparison between Tornado Cash, Aztec, and our protocol.

Scheme	Deposit/withdrawal	Viewing key	Deposit gas	Transfer gas	Withdraw gas
Tornado Cash [17]	Fixed	No	972795	N/A	329388
Aztec [19]	Flexible	Permanent	1088600	938471	969751
Our protocol	Flexible	Revocable	276395	209760	244582

Although private transactions aren't natively supported, our protocol can be deployed on public blockchains instead of forking to private chains like Hyperledger [4] and ZCash [5]. Its feasibility was shown in experiments on EVM-based blockchains. User experience can be enhanced with front-end solutions that simplify balance management and interactions, making private transactions as easy as regular ones.

We compare our protocol with Tornado Cash and Aztec, both deployed on the Ethereum mainnet, as summarized in Table 5. For the Deposit/Withdrawal criterion, Tornado Cash's reliance on Merkle tree leaves limits it to fixed amounts, while Aztec permits arbitrary deposits and withdrawals if users prove equal total amounts in a joint-split transaction. Our approach, using

homomorphic ciphertexts, allows flexible transactions by tracking user balances with the ability to add and subtract arbitrary values.

About the ability for users to reveal their balances to third parties, Tornado Cash requires users to disclose their nullifier and secret value, making it risky to share. In contrast, Aztec allows safe sharing of viewing keys while keeping spending keys secret, but its viewing keys cannot be rotated or revoked, permanently exposing the balance. Our protocol enhances this by allowing viewing key rotation through the ProveRotate algorithm.

About gas costs, Tornado Cash updates its Merkle tree per deposit¹ and verifies a Merkle proof per withdrawal², resulting in logarithmic complexity. Aztec's joint-split transactions^{3,4,5} require linear complexity for proof verification. In contrast, our protocol uses zkSNARK proof verification, achieving constant complexity. Our deposit transactions cost approximately 280K gas, reducing costs by 71.6% compared to Tornado Cash and 74.6% compared to Aztec. Withdrawal transactions require half the gas of Tornado Cash and one-fourth that of Aztec. For fund transfers, Tornado Cash does not support this, and Aztec costs nearly 1 million gas, while our protocol only requires around 240K gas, about one-fifth of Aztec's cost.

Finally, we discuss about trusted setup requirement. Both Tornado Cash and our protocol offer flexibility in proof system selection, balancing computational cost and transparency. In contrast, Aztec's customized proof system necessitates a trusted setup. Our concrete construction uses Groth16 for efficient verification, with this trusted setup being a one-time process executed via multi-party computation to eliminate any toxic waste.

5 CONCLUSION

This work demonstrates how a ZKP-based protocol can ensure confidentiality for transactions on a public blockchain. By using encrypted balances and zkSNARK proofs, sensitive information remains hidden in a public ledger. The design supports various cryptographic primitives, allowing flexibility in choosing encryption or commitment schemes based on performance and trust assumptions. This modular approach, paired with off-chain execution and constant-size commitments, enables scalability without bloating on-chain storage. While hardware acceleration could improve proof-generation, our experiments indicate that succinct on-chain verification remains highly efficient. Thus, this work lays the groundwork for integrating blockchain technology into financial systems that require strong privacy.

Acknowledgments

This work was supported by the “Scientific Research and Innovation Program” of Ho Chi Minh University of Technology under the grant No. SVKSTN-2024-KH&KTMT 56. The authors acknowledge Ho Chi Minh City University of Technology (HCMUT), VNU-HCM, for supporting this study.

References

- [1] Ronald Cramer, Rosario Gennaro, and Berry Schoenmakers. 1997. A secure and optimally efficient multi-authority election scheme. *Eur. Trans. Telecommun.* 8, 5 (1997), 481–490. doi:10.1002/ETT.4460080506
- [2] John R. Douceur. 2002. The Sybil Attack. In *Peer-to-Peer Systems*, Peter Druschel, Frans Kaashoek, and Antony Rowstron (Eds.). Springer, Berlin, Heidelberg, 251–260. doi:10.1007/3-540-45748-8_24
- [3] Belen Sağlam et al. 2020. A Data-Driven Analysis of Blockchain Systems' Public Online Communications on GDPR. In *2020 IEEE International Conference on Decentralized Applications and Infrastructures (DAPPS)*. 22–31. doi:10.1109/DAPPS49028.2020.00003
- [4] Elli Androulaki et al. 2018. Hyperledger fabric: a distributed operating system for permissioned blockchains. In *Proceedings of the Thirteenth EuroSys Conference, EuroSys 2018, Porto, Portugal, April 23-26, 2018*, Rui Oliveira, Pascal Felber, and Y. Charlie Hu (Eds.). ACM, 30:1–30:15. doi:10.1145/3190508.3190538
- [5] Eli Ben-Sasson et al. 2014. Zerocash: Decentralized Anonymous Payments from Bitcoin. In *2014 IEEE Symposium on Security and Privacy*. 459–474. doi:10.1109/SP.2014.36
- [6] Jung Hee Cheon et al. 2017. Homomorphic Encryption for Arithmetic of Approximate Numbers. In *Advances in Cryptology – ASIACRYPT 2017*, Tsuyoshi Takagi and Thomas Peyrin (Eds.). Springer, Cham, 409–437. doi:10.1007/978-3-319-70694-8_15
- [7] Jiawen Kang et al. 2019. Blockchain for Secure and Efficient Data Sharing in Vehicular Edge Computing and Networks. *IEEE Internet Things J.* 6, 3 (2019), 4660–4670. doi:10.1109/JIOT.2018.2875542
- [8] Meng Zhang et al. 2019. Protecting Data Privacy for Permissioned Blockchains using Identity-Based Encryption. 602–605. doi:10.1109/ITNEC.2019.8729244
- [9] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. 2019. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. *IACR Cryptol. ePrint Arch.* (2019), 953. <https://eprint.iacr.org/2019/953>
- [10] Jens Groth. 2016. On the Size of Pairing-Based Non-interactive Arguments. In *Advances in Cryptology – EUROCRYPT 2016*, Marc Fischlin and Jean-Sébastien Coron (Eds.). Springer, Berlin, Heidelberg, 305–326. doi:10.1007/978-3-662-49896-5_11
- [11] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. 2010. Constant-Size Commitments to Polynomials and Their Applications. In *Advances in Cryptology – ASIACRYPT 2010*, Masayuki Abe (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 177–194. doi:10.1007/978-3-642-17373-8_11
- [12] Greg Maxwell. 2013. *CoinJoin: Bitcoin privacy for the real world*. Retrieved September 15, 2024 from <https://bitcointalk.org/?topic=279249>
- [13] Ralph C. Merkle. 1988. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology – CRYPTO '87*, Carl Pomerance (Ed.). Springer, Berlin, Heidelberg, 369–378. doi:10.1007/3-540-48184-2_32
- [14] Satoshi Nakamoto. 2008. *Bitcoin: A Peer-to-Peer Electronic Cash System*. Retrieved December 12, 2024 from <https://bitcoin.org/bitcoin.pdf>
- [15] Pascal Paillier. 1999. Public-Key Cryptosystems Based on Composite Degree Residuosity Classes. In *Advances in Cryptology – EUROCRYPT '99*, Jacques Stern (Ed.). Springer, Berlin, Heidelberg, 223–238. doi:10.1007/3-540-48910-X_16
- [16] Torben Prids Pedersen. 1992. Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing. In *Advances in Cryptology – CRYPTO '91*, Joan Feigenbaum (Ed.). Springer, Berlin, Heidelberg, 129–140. doi:10.1007/3-540-46766-1_9
- [17] Alexey Pertsev, Roman Semenov, and Roman Storm. 2019. *Tornado Cash Privacy Solution*. Retrieved October 5, 2024 from <https://berkeley-defi.github.io/assets/material/Tornado%20Cash%20Whitepaper.pdf>
- [18] Ronald L. Rivest, Adi Shamir, and Yael Tauman. 2001. How to Leak a Secret. In *Advances in Cryptology – ASIACRYPT 2001*, Colin Boyd (Ed.). Springer, Berlin, Heidelberg, 552–565. doi:10.1007/3-540-45682-1_32
- [19] Zachary J Williamson. 2018. *The aztec protocol*. Retrieved September 30, 2024 from <https://github.com/AztecProtocol/AZTEC>

¹<https://etherscan.io/tx/0x5053edc81c29fda2e1a304aa2fb096a0a53d0551b1b6eaffbbe829b0ac7d22dc>

²<https://etherscan.io/tx/0x8a161d653a5d4b243fa261d0b9ea83edd7c44677015e9c963a01ecd57906b320>

³<https://etherscan.io/tx/0xc25ee135911ed0ab68e1d63fad6abd1df7b11fc787c6044995a16dcad8f9bb8>

⁴<https://etherscan.io/tx/0x6cb6bcb6d51445ce026dd76b8526e8014a6a276255d22e4f5be26f8efb891fb>

⁵<https://etherscan.io/tx/0xf9a101682c637f7741f281c858527d17036f4df284b7064bd1ca44531ab88374>